Forecasting Floods with a CNN-LSTM

A plain-language guide to the project: what we did, why, what came out, and what is still weak

DL Lab project · Anhad Mahajan & Aman Sumeet Saxena

Corrected version, 4 October 2026. It adds a simple "trend" forecast to compare with, a second version of the forecaster (version 2), and corrects several statements of the first guide.

> **The whole project in one paragraph**
>
> We teach a computer program to look at maps of a flood and predict where the water will be **30 minutes later**. The program learns from four floods (Australia, the UK, Pakistan and Mozambique). We test it on a flood it has **never seen** and compare it with two very simple guesses: "the water stays exactly where it is now" and "the water keeps changing the way it just changed". On top of the main program we added three extras the course asks for: a **capsule network** that rates how bad the flooding is, **Grad-CAM** pictures that show where the program is looking, and **written rules** (the "NeuroSymbolic" part) that check the program's answers with common sense and physics.

> **What changed in this corrected version**
>
> 1. We added a second simple guess, the **trend forecast**. It beats the first version of our CNN-LSTM in every test (it scores 0.91 to 0.99; the first version scored 0.15 to 0.35, and below zero for late Australia).
> 2. We built **version 2** of the forecaster. It starts from the trend forecast and learns only a correction on top. It **matches** the trend forecast but does not beat it, because at 30 minutes ahead there is almost nothing left to learn.
> 3. The explanation we gave for the late-Australia failure was wrong. The first version added a little water over most of the map. The training part of the flood did contain falling water (in 73 of its 331 steps the average depth fell), and the trend forecast fitted on it does very well on the last 15%, so missing examples of draining water were not the cause.
> 4. The water-balance rule helped the first version but not version 2, so it is now described as a physics check, not an improvement. The data follows the water balance closely (within about 5%), not "exactly".
> 5. Statements about what the model "learned about floods", about the LSTM and about the Grad-CAM pictures were toned down to what the tests actually show.
> 6. The capsule network and the Grad-CAM pictures are still those of the **first version**. We have not rebuilt them on version 2.

Contents

1. The problem and the data
2. Preparing the data
3. The yardsticks: two simple forecasts
4. The main model: CNN + LSTM
5. How the models were trained and tested
6. Results: do they beat the yardsticks?
7. The water-balance rule (physics check)
8. The capsule network: how bad is the flood?
9. The severity rules (NeuroSymbolic part 2)
10. Grad-CAM: where is the model looking?
11. What went wrong along the way, and how we fixed it
12. Honest weaknesses
13. Next steps
14. Questions the professor may ask, with simple answers
15. Word list

1. The problem and the data

Why forecast floods?

When a river floods, people need to know **where the water is going next**: which roads, fields and houses will be under water soon. Forecasting even a short time ahead gives time to warn people and move them.

Where the data comes from

We use a public dataset called **FloodCastBench** (Xu and others, published in the journal *Scientific Data*, 2025). The authors took four real floods and re-created them in a **computer simulation**: a program that uses the laws of how water flows (water runs downhill, rain adds water, rough ground like forest slows it down).

> **Important to say clearly**
>
> The flood maps are **simulated**, not photos from satellites. The simulation is built from real ground heights and real satellite rainfall, so it is realistic, but it is still a simulation. This is normal for research like this, because real measurements of water depth every few minutes over a whole region do not exist.

Flood	When	Size of one map square (cell)	Map size	Length
Australia	Feb – Mar 2022	30 m × 30 m	1073 × 1073 cells (about 32 km wide)	10 days
UK	Dec 2015	30 m × 30 m	170 × 275 cells	3 days
Pakistan	Aug 2022	480 m × 480 m	810 × 441 cells (about 390 km tall)	14 days
Mozambique (Cyclone Idai)	Mar 2019	480 m × 480 m	151 × 138 cells	6 days

Think of each map as a **grid of squares**, like a chessboard with a million squares. For each square we know:

- **Water depth**: how many metres of water are on that square. This is what we want to predict.
- **Rain**: how much rain is falling there (millimetres per hour).
- **Ground height**: how high the ground is. Water flows from high squares to low squares.
- **Land cover**: water, trees, crops, buildings and so on.

We have one map every **30 minutes**, so each flood is like a flip-book of hundreds of pictures.

> **How the coarser maps were made**
>
> The dataset gives a flood at one or two map sizes. According to the dataset paper, the 60 m maps of Australia and the UK are the 30 m simulation results resized to a coarser grid. For Pakistan, the authors shrank the terrain and rainfall to 480 m and ran the simulation again at that size, so its 480 m maps are not resized copies of 30 m results. The paper does not say which of the two methods was used for Mozambique, so we do not claim either.

2. Preparing the data

Before any learning, the raw files had to be cleaned and lined up. This was done in our earlier notebooks (*Data_preparation* and the pre-processing and visualisation notebook).

1. **One map every 30 minutes.** The dataset has depth maps every 5 minutes, but rain only every 30 minutes, and water barely moves in 5 minutes. So we keep one depth map in six.
2. **Put every layer on the same grid.** The ground-height and rain files come at a finer size (30 m) than some depth maps (480 m). We averaged the small squares inside each big square, so every layer describes exactly the same place.
3. **Check the layers line up.** If a layer were shifted by even a few squares, the program would learn from wrong information. We checked that rivers on the land-cover map and valleys on the ground-height map sit where the water collects, and the best-matching positions agree with the ones we use to within about one cell.
4. **Use rain honestly.** Each rain file holds the rain that falls in the **next** 30 minutes. A forecast is not allowed to know the future, so the program only ever sees rain that has **already fallen**.
5. **Save to Google Drive**, so the data loads in seconds instead of downloading 27 GB again.

How rain lines up with the maps

```
map at 1:00         map at 1:30          map at 2:00
      |---- rain A ----|---- rain B ----|
```

To forecast the 2:00 map while standing at 1:30, the program may use rain A (it has already fallen) but **not** rain B (it is still in the future).

3. The yardsticks: two simple forecasts

To know if a forecast is any good, we need something to compare it with. We use two very simple forecasts that need no learning.

Yardstick 1: "copy the last map"

"In 30 minutes the water will be exactly where it is now." It costs nothing and needs no learning.

Yardstick 2: the "trend forecast"

"In 30 minutes the water will have changed the way it changed in the last 30 minutes." It is like guessing tomorrow's temperature: if it got 2 degrees warmer between yesterday and today, you guess about 2 degrees warmer again. For each square, the forecast is the depth now plus the last change in depth times one number. That number says how much of the last change continues. It is 0.92 for the 30 m floods and 0.95 for the 480 m floods, and it was found using only the first part of the training flood, so the trend forecast knows nothing about the test floods.

Over 30 minutes a flood changes only a little, so even "copy the last map" is already **very good**. An earlier version of this project reported impressive-looking numbers without comparing to this yardstick; when compared, that version was barely better than copying. In the first version of this guide every result was shown only next to "copy the last map". A later check showed that this was too easy a yardstick, because the trend forecast is much better than copying. So every result is now shown next to both.

The score we use: skill

Skill value	Meaning
1	Perfect forecast
between 0 and 1	Better than copying. For example 0.3 means the forecast removes 30% of copying's error
0	No better than copying
below 0	Worse than copying

In formula form: skill = 1 − (forecast's error ÷ copying's error), where "error" is the sum of the squared differences between forecast and truth over every square. Skill is always measured against copying, so a skill of 0.9 for the trend forecast means it removes 90% of copying's error.

4. The main model: CNN + LSTM

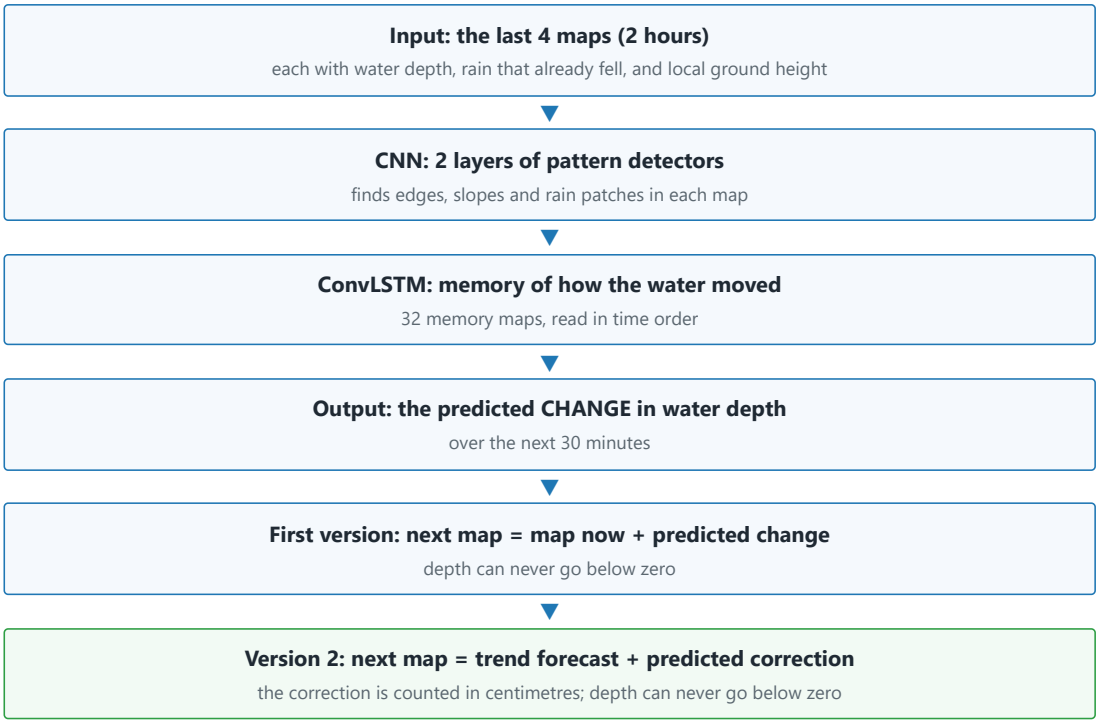
The model has two parts, each with a job a person would also do when reading flood maps.

Part 1: the CNN looks at each picture

A **CNN** (convolutional neural network) is the standard tool for pictures. It slides small 3 × 3 "windows" across the map and learns to spot patterns: the edge of the water, a slope, a valley, a patch of heavy rain. The same windows are used everywhere on the map, so a valley is recognised wherever it is.

Part 2: the LSTM remembers what happened

An **LSTM** is a network with **memory**. It reads the maps in order (2 hours ago, 1.5 hours ago, 1 hour ago, now) and keeps track of how the water has been moving: is it rising, spreading, draining? We use a **ConvLSTM**, an LSTM whose memory is itself a map, so every square remembers what happened around it. A normal LSTM would turn the map into a long list of numbers and lose the shape of the land.



Three design choices

- **Predict the change, not the whole map.** Most of the map hardly changes in 30 minutes. If the model had to redraw the whole map, it would waste effort on what it already knows. Predicting only the change makes it focus on where water moves. The model also **starts** as "copy the last map" (its last layer begins at zero), so training can only improve on that yardstick.
- **Local ground height instead of height above sea level.** Water doesn't care how high a field is above the sea; it cares whether the field is **higher or lower than its neighbours**. So we give the model "how much higher or lower than the surroundings" for each square.
- **Version 2 starts from the trend forecast.** The first version starts from "copy the last map" and has to learn everything else. Version 2 starts from the trend forecast and learns only a small correction on top of it. Its last layer starts at zero, so before any training it *is* the trend forecast (we checked this: the largest difference was 0.0).

> **Why version 2 exists**
>
> After we found that the trend forecast beats the first version (see section 6), we built version 2 to see whether the same network could at least match it. It has the same 62,833 numbers to learn. We changed two things at once, the starting point and counting the correction in centimetres, so we cannot say which of them made the difference.

The model is small: about **63,000 numbers** to learn (a phone photo has millions of pixels). Small is good here: it trains in minutes and is less likely to just memorise the training flood.

5. How the models were trained and tested

Two models, one per map scale

A 30 m square and a 480 m square behave very differently (water crosses a 30 m square in seconds, a 480 m square in minutes). So we train two separate models for each version:

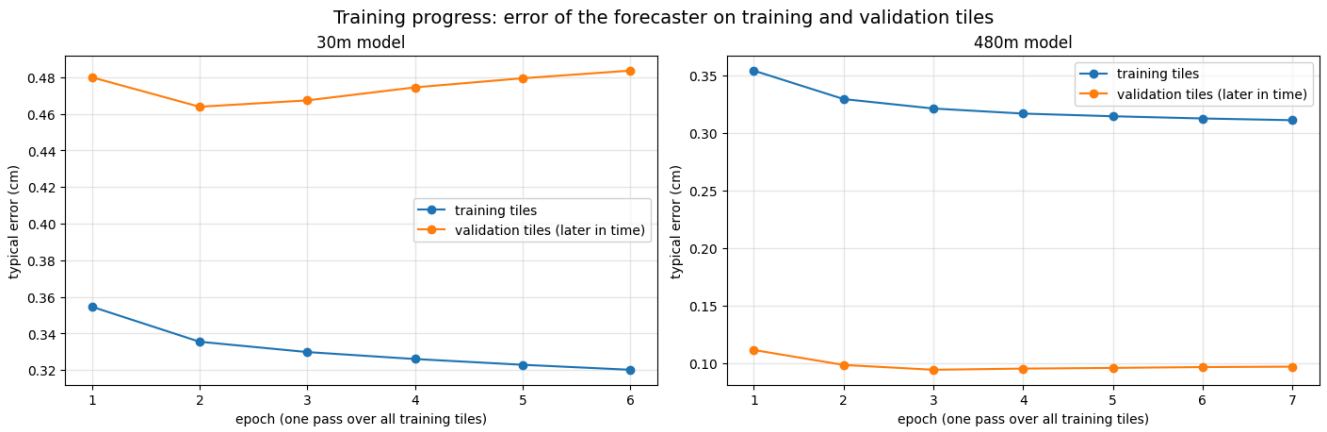
Model	Learns from	Tested on
30 m model	Australia, first 70% of the flood	Australia, last 15% **and** the whole UK flood (never seen)
480 m model	Pakistan, first 70% of the flood	Pakistan, last 15% **and** the whole Mozambique flood (never seen)

The middle 15% of each training flood is used as a **check during training** ("validation"): if the model gets better on the training part but worse on this check, it is starting to memorise, and training stops.

> **Why test on a never-seen flood?**
>
> Testing on a flood the model never saw is the honest test. It shows whether the model learned **how floods behave in general** rather than memorising one place. We only have one such test flood for each map scale, so the results rest on two training-and-test pairs.

Tiles

The Australia map has over a million squares, too big to learn from all at once. So the model learns from **4,000 small square pieces (tiles) of 64 × 64 squares**, cut at random places and times. Because the model only uses small sliding windows, the same learned model can afterwards be run on a **whole map**, which is how all results are measured.



Training progress of the first version. The error on the training tiles (blue) and on the later "check" tiles (orange), after each pass over the data. Training stopped automatically when the check error stopped improving, after only a few passes. Judging from the figure, the 30 m model kept the weights from its second pass, because its check error got worse after that.

Version 2 was trained the same way, on the same tiles. On the check tiles its error before training (the plain trend forecast) was 0.1885 cm for the 30 m model and 0.0243 cm for the 480 m model. After training it was 0.1868 cm and 0.0230 cm, so training only changed the forecast a little. All results in this guide come from **a single training run** per model.

6. Results: do they beat the yardsticks?

Flood	Tested on	Copy the last map: typical error	Skill: trend forecast	Skill: first version	Skill: version 2
Australia	unseen last 15%	0.11 cm	**+0.990**	**−1.449**	**+0.989**
UK	never-seen flood	0.52 cm	**+0.947**	**+0.305**	**+0.948**
Pakistan	unseen last 15%	0.10 cm	**+0.908**	**+0.345**	**+0.908**
Mozambique	never-seen flood	0.55 cm	**+0.942**	**+0.150**	**+0.943**

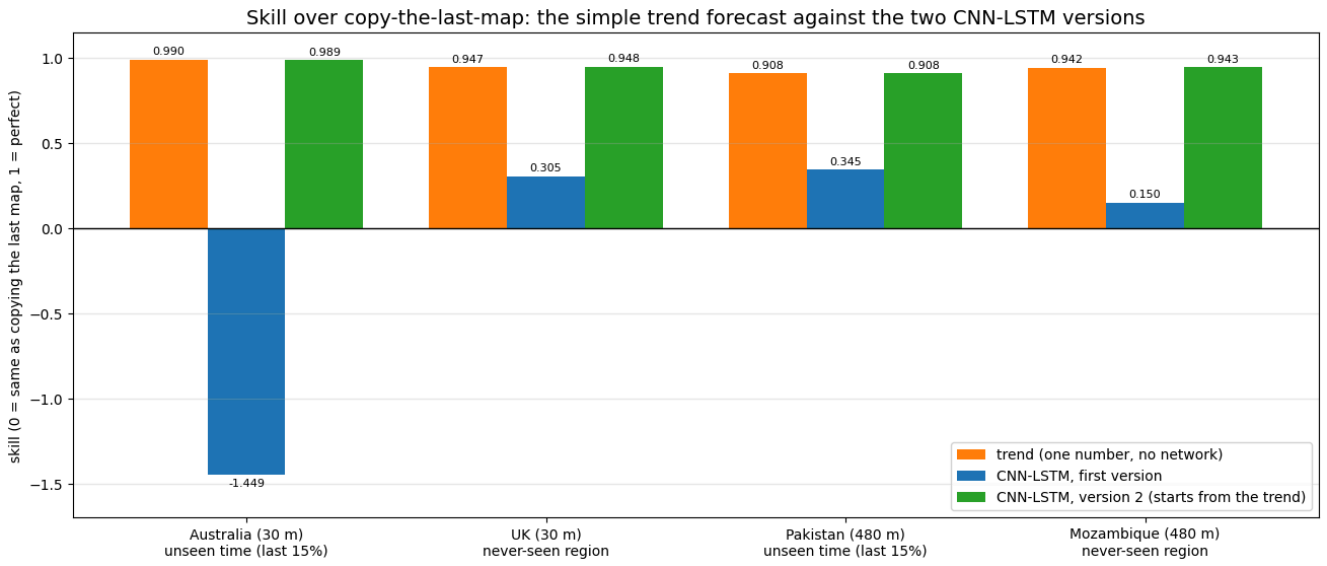
Skill is measured against "copy the last map" (section 3). The "typical error" of copying is averaged over the whole map, dry squares included, so over wet squares only it would be higher (for copying, the whole-map number is about 20% to 45% lower than the wet-squares number).

> **The main result**
>
> On the two floods the models have never seen, the first version beat "copy the last map" (skill +0.31 in the UK and +0.15 in Mozambique). But the trend forecast, with no network at all, scored +0.95 and +0.94. Version 2, which starts from the trend forecast, matches it (+0.95 and +0.94). So the forecasters are useful in the sense that they beat copying, and version 2 is as good as the strongest simple forecast we tried, but it is not yet better than it.

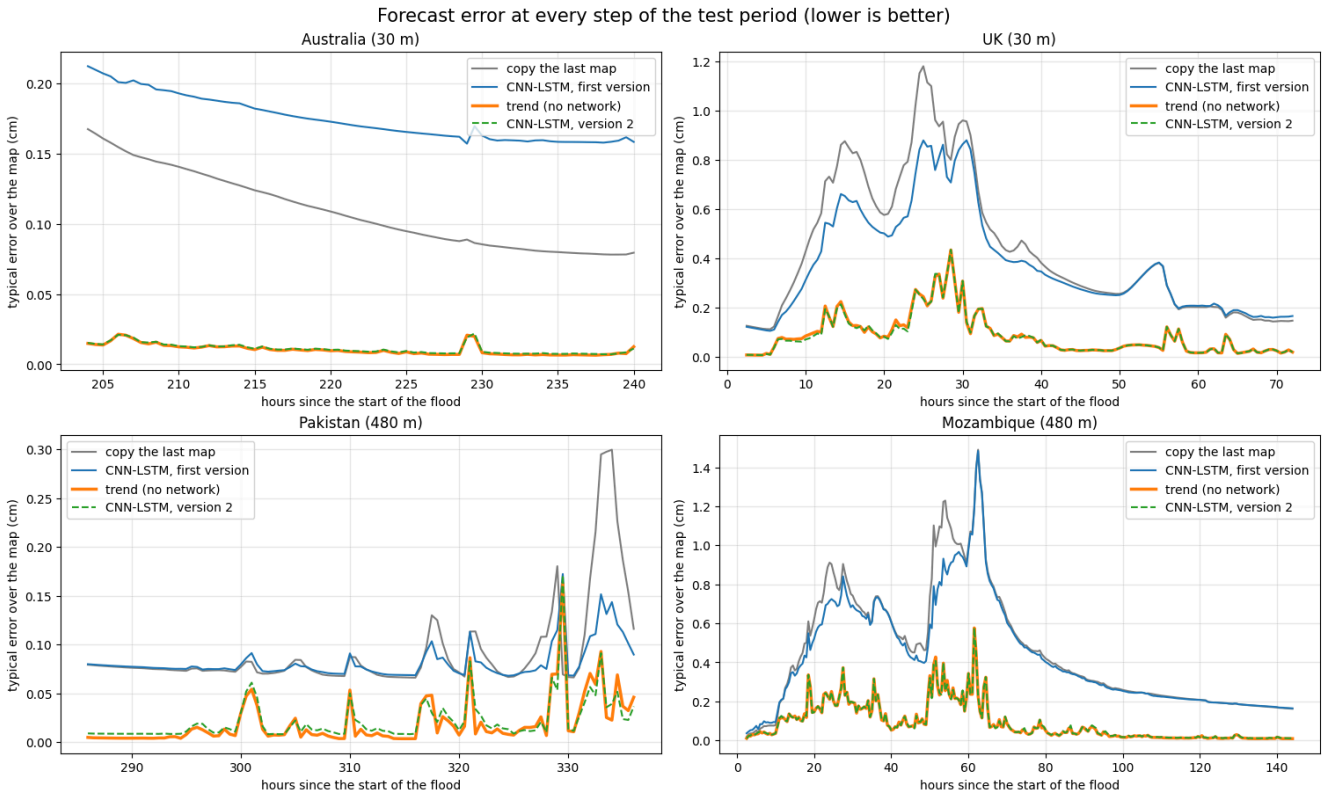
> **How much is left to learn**
>
> At 30 minutes ahead the trend forecast already removes 91% to 99% of copying's error. So the most any model could still add is about 1% (Australia), 5% (UK), 9% (Pakistan) and 6% (Mozambique) of copying's error. Version 2 differs from the trend forecast by 0.001 or less in every test, so at this horizon the network adds almost nothing to the trend.

The first version's failure: late Australia

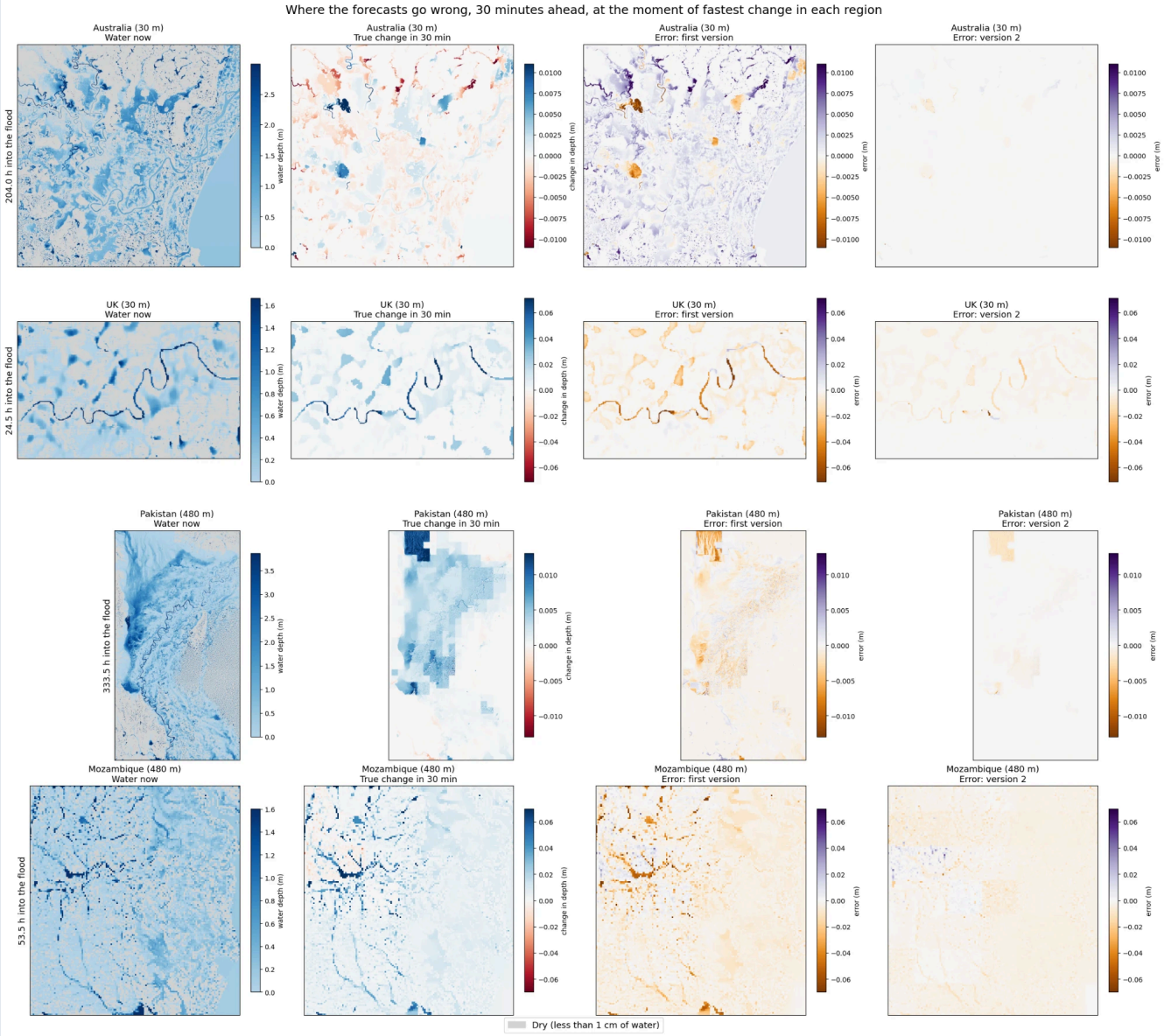
At the end of the Australian flood the water is almost still: copying is wrong by only about 1 millimetre. The first version scored −1.45 there, worse than copying. We first explained this by saying that the model learned only from the rising, rainy part of the flood and never saw water draining away. That explanation does not hold. In 73 of the 331 training steps the average depth of the Australian map fell, so falling water was in the training data, and the last 15% is a very quiet period in which the average depth still rises slightly in every step. The trend forecast, fitted on the same training steps, scores +0.99 on that last part. The error maps below show what really went wrong: the first version predicts slightly too much water over most of the map, and misses the local rises and falls. Version 2 scores +0.989 there.



Skill per flood. Bars above the black line mean better than copying; 1 is perfect. Orange is the trend forecast (no network), blue the first version, green version 2.



Error at every moment of the test period (lower is better). Grey is copy the last map, blue the first version, orange the trend forecast, green dashed version 2. The orange and green lines lie almost on top of each other and far below the others. In the calm early part of Pakistan's test period version 2 is very slightly above the trend forecast, so version 2 matches the trend and does not beat it. In Australia the first version's error stays between 0.16 and 0.21 cm while copying's error falls to about 0.08 cm, which is what an extra bias that does not go away looks like.



Where the forecasts go wrong, one row per flood, at the moment of fastest change. Columns: water now, true change in 30 minutes, error of the first version, error of version 2. In the error columns **purple means the forecast had too much water and orange means too little**; the last three columns share one colour scale. The first version's errors are strong along the rivers and rain blocks, and in Australia there is a faint purple haze over most of the map. Version 2's errors are almost invisible at the same scale.

7. The water-balance rule (physics check) NeuroSymbolic part 1

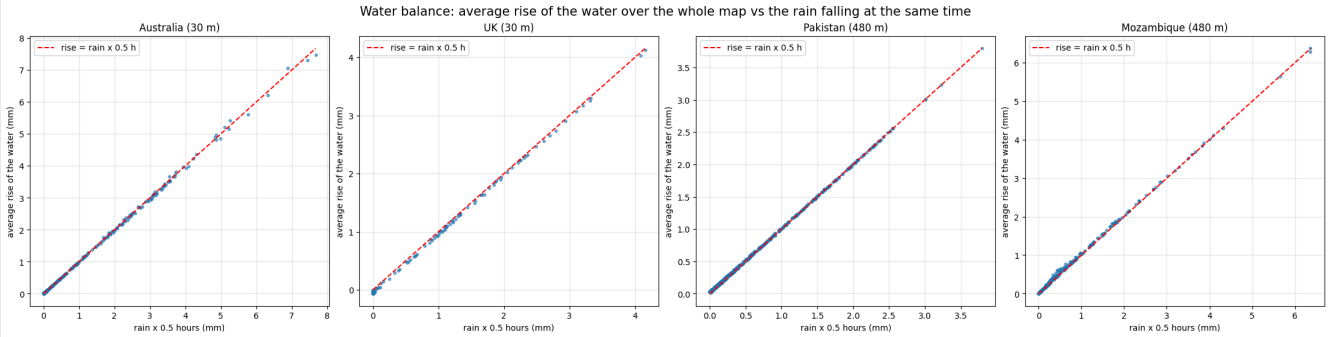
"NeuroSymbolic" means combining a **neural network** (which learns from examples) with **written rules** (which a person can read and check). Our first rule comes from physics.

The law

In these simulations almost all of the rain stays on the ground, so the rise of the water matches the rain. According to the dataset paper, the Australian, UK and Mozambique simulations let water leave freely at the edge of the map, and the Pakistan simulation has a river flowing in. So some water does cross the edge, and the law below is only an approximation. If no water crossed the edge, then over the whole map:

> **average rise of the water in 30 minutes = rain (mm per hour) × 0.5 hours**
> With no rain, the total amount of water cannot change. It can only move around.

We first checked how well the data follows this law:



Water-balance check. Each dot is one 30-minute step. Across: rain × 0.5 hours. Up: how much the water rose on average. The dots sit very close to the red line in all four floods.

Flood	Australia	UK	Pakistan	Mozambique
Total rise of the water ÷ total (rain × 0.5 h)	0.999	0.946	1.028	1.045

So over a whole flood the water added differs from the rain by up to about 5%. The law holds **closely, not exactly**. Water crossing the edge of the map is a likely reason, and in 73 of the 331 training steps of the Australian flood the average depth even fell, which cannot happen from rain alone. The rise matches the rain of the **same** 30 minutes almost perfectly (correlation 1.00), and the rain of the 30 minutes **before** less well (0.93 to 0.97).

What the rule does

The neural network does not know this law, and the first version had picked up a small habit of adding a little water everywhere. After every forecast, the rule:

1. adds up all the water the network predicts for the whole map;
2. compares it with the amount the rain allows. It can only use rain that has **already fallen**, so it uses the last 30 minutes of rain as its guess for the next 30 minutes. That is the only honest choice, but it is an approximation;
3. if the network added too much, removes the extra from wet squares; if too little, spreads the missing rain over every square.

Where the water goes is still decided by the network. The rule only fixes the **total**.

Flood	Copy + rule (no network)	First version	First version + rule	Trend forecast	Version 2	Version 2 + rule
Australia	0.000	**−1.449**	**−0.710**	0.990	0.989	0.989
UK	0.053	0.305	0.320	0.947	0.948	0.949
Pakistan	0.142	0.345	0.404	0.908	0.908	0.901
Mozambique	0.051	0.150	0.172	0.942	0.943	0.941

All numbers are skill against copying the last map. The trend-forecast-plus-rule scores are within 0.003 of the trend forecast alone.

> **What this tells us about the rule**
>
> The rule helped the first version in every flood and halved Australia's problem, because that version added extra water. On version 2 the rule makes almost no difference (between −0.007 and +0.001) and makes Pakistan slightly worse. So we keep the rule as a physics check, not as an improvement. It is still a fair example of NeuroSymbolic AI: the network spots where the water goes, and a written rule makes sure the total is sensible. It just does not raise the score once the forecaster is good.

8. The capsule network: how bad is the flood?

The question it answers

Besides the map, people want a simple rating: "how bad will it be here in 30 minutes?". We cut the map into tiles of 64 × 64 squares and rate each tile by the **share of its normally-dry land under more than 30 cm of water** (rivers and lakes don't count):

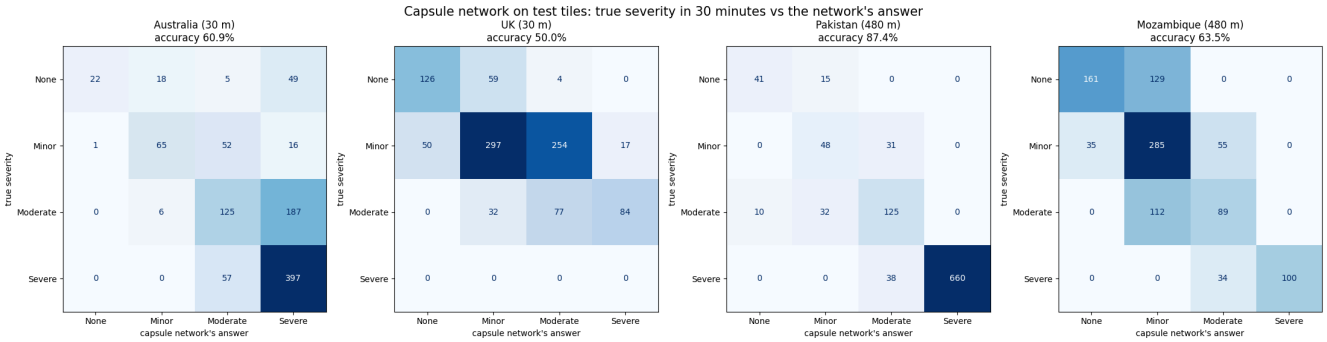
Rating	Share of dry land under 30 cm+ of water
None	less than 5%
Minor	5% – 15%
Moderate	15% – 30%
Severe	30% or more

30 cm is roughly the depth at which water becomes dangerous to walk or drive through.

What a capsule network is

A normal network ends with **one number per answer** ("70% sure it's Severe"). A capsule network ends with a small **arrow (a list of numbers) per answer**: the **length** of the arrow says how sure it is, and the **direction** of the arrow stores **how** the answer looks (where the water is, how it is spread out).

Inside, many small capsules each look at one part of the tile and "vote" for each rating. Votes that **agree** with each other get more weight in the next round ("routing by agreement"). It is like a group discussion where people who agree with the emerging opinion get listened to more. Our capsule network reuses the CNN + LSTM of the **first version** of the forecaster (frozen, so it already understands flood maps) and adds the capsule layers on top.



How often the capsule network is right, one grid per flood. Rows: the true rating. Columns: the network's answer. Numbers on the diagonal (top-left to bottom-right) are correct answers. Accuracy: Pakistan 87%, Mozambique 64%, Australia 61%, UK 50%.

> **Honest assessment**
>
> The capsule network works, but it is the weakest part. The rating in 30 minutes is almost the same as the rating **now**, and if you simply apply the rating rule to the forecaster's predicted map, you get it right about 99% of the time (99.8%, 99.1%, 99.9% and 99.5% in the four floods). So the capsule network is doing a hard version of an easy job. Three more cautions:
>
> - In the Pakistan test 698 of the 1,000 tiles are Severe, so always answering "Severe" would already be right 69.8% of the time, and 87% accuracy is less impressive than it looks.
> - A 64 × 64 tile covers about 2 km at 30 m but about 30 km at 480 m, so "Severe" means different things in the two models, and their accuracies should not be compared with each other.
> - We have not compared the capsule network with a plain output layer. In our earlier, larger laptop study (the "round 2" study) the capsule network only beat a plain layer when no data augmentation was used.

9. The severity rules NeuroSymbolic part 2

The second set of written rules checks the capsule network's rating. Each final answer comes with a **written reason**, so a person can see why the decision was made.

Rule	If…	Then…
1. No water, no flood	less than 1% of the land is flooded now **and** in the forecast	rating = None
2. Rising water	the forecast shows at least as much flooding as now **and** it rained more than 1 mm/h in the last hour	rating can't be lower than the current level
3. Agree with the forecast	the capsule network's rating is more than one level away from the rating of the forecast map	use the forecast map's rating

Flood	Capsule network alone	Capsule network + rules	Answers changed by the rules
Australia	60.9%	**67.9%**	7%
UK	50.0%	**56.1%**	6%
Pakistan	87.4%	**89.1%**	2%
Mozambique	63.5%	**70.7%**	7%

The rules improve the rating in every flood, by up to 7 percentage points. Example of a written reason from the results: *"Rule 2: water rising during rain, so at least Moderate (the current level)."*

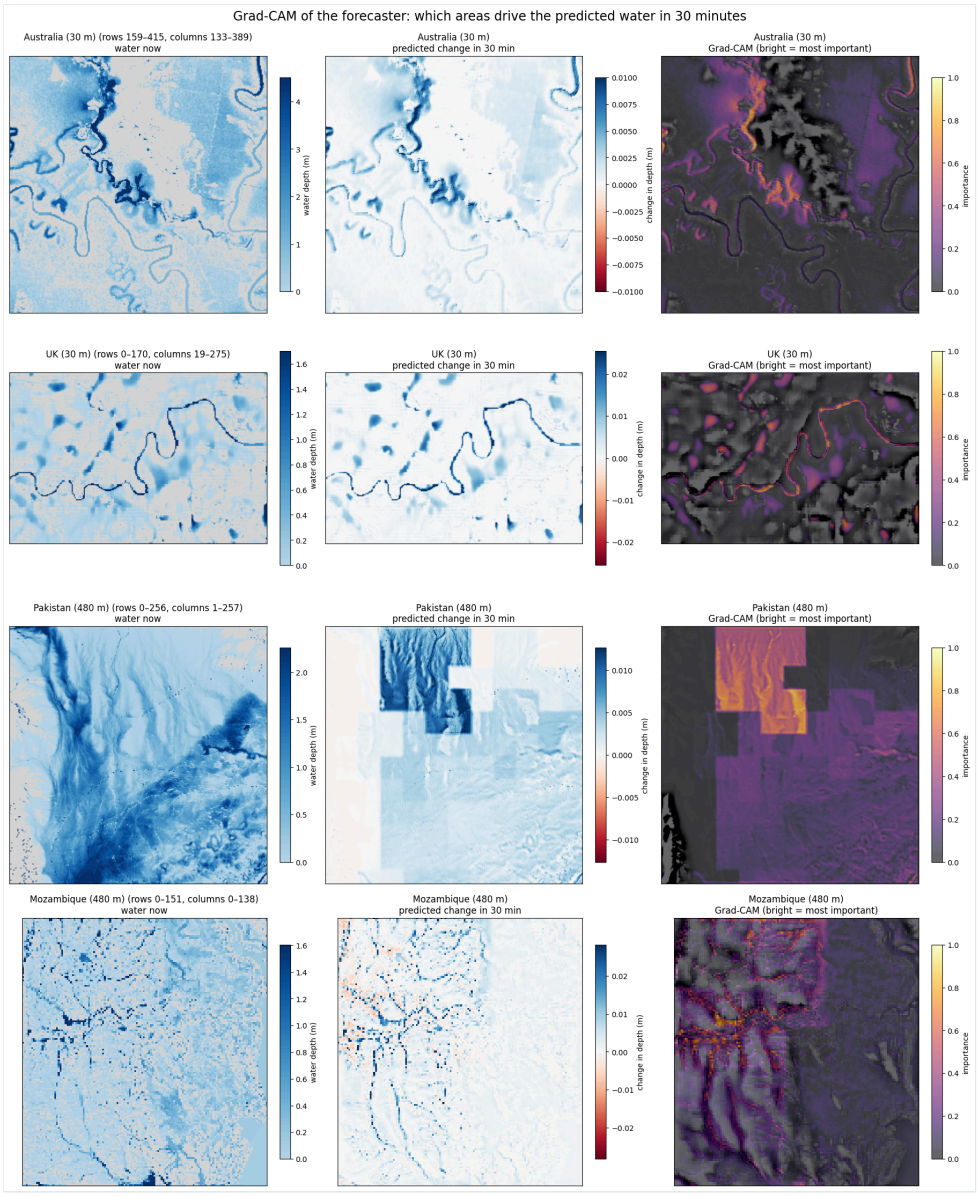
> All three rules use the flooded share of the map now or in the forecast, which is almost always a good guide to the rating. Rules 1 and 2 did most of the work (rule 1 fired 134 times in Mozambique and rule 2 fired 40 times in the UK), and rule 3 replaced the capsule rating by the forecast map's rating in 21, 21 and 10 tiles in Australia, the UK and Pakistan. So the gain says more about the rules than about the capsule network.

10. Grad-CAM: where is the model looking?

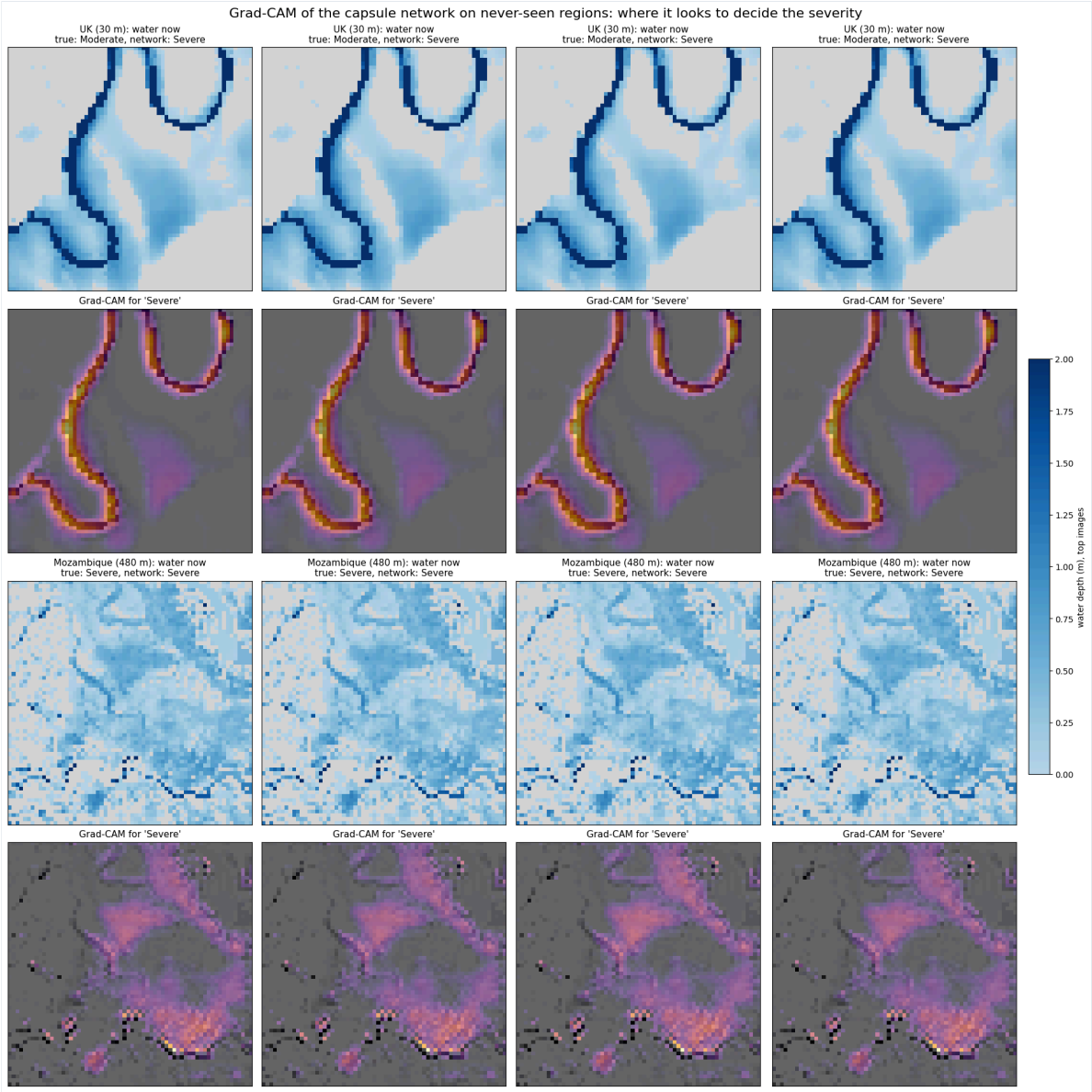
Neural networks are often called "black boxes". **Grad-CAM** opens the box a little: it makes a **heat map** showing which parts of the map most influenced the answer. Bright means very important, dark means ignored.

How it works, in plain words: we ask "if this part of the model's memory changed a little, how much would the forecast change?". Parts that would change the forecast a lot are important, and are painted bright.

> **What the heat maps show**
>
> - **Australia and UK:** the bright areas are the river channels and flood edges, where water is actually changing.
> - **Pakistan:** the bright areas are the square patches of heavy rain.
> - **Mozambique:** the bright areas follow the drainage lines where water collects.
>
> These are sensible places. But the heat maps are **illustrations, not proof**. In our earlier laptop study, a check of the capsule network's heat maps found that they changed a lot from one training run to another, and we have not run such a check on the pictures in this guide.



Grad-CAM of the forecaster (first version). Columns: water now, predicted change, heat map (on top of the ground shape). For the large maps we zoom into the busiest area, because a whole map does not fit in the graphics card's memory.



Grad-CAM of the capsule network on never-seen floods. Top of each pair: the water. Bottom: where the network looked to decide the rating. In the UK it looks at the river and the flooded field beside it; in Mozambique at the large flooded patches. *Note:* the four UK examples (and the four Mozambique ones) are the same place at nearly the same moment, so they look identical; the next version should pick more varied examples.

11. What went wrong along the way, and how we fixed it

These are worth mentioning: they show the results were checked, not just accepted.

> **Problem 1: the model made up water (first run: Australia skill −12.7)**
>
> To help the model learn from interesting places, the first version kept only 20% of the "calm" tiles where nothing changes. The result: the model almost never saw "no rain, nothing happens", and learned that **water always rises a little**. It added water everywhere, even over the sea. **Fix:** keep all tiles. Australia improved from −12.7 to −1.45, and calm periods in the other floods stopped losing to copying.

> **Problem 2: out of memory when drawing Grad-CAM**
>
> Grad-CAM must keep every intermediate step in the graphics card's memory, and the full Australia map (over a million squares) did not fit. **Fix:** zoom into a 256 × 256 window around the area of fastest change.

Problem 3: the first water rule hurt Mozambique

At the start of the Mozambique flood the map was almost dry. The first version of the rule piled all the missing rain onto the few wet squares. **Fix:** rain falls everywhere, so missing rain is now spread over every square. Mozambique went from +0.03 back to +0.17.

Problem 4: the first version was far below a simple trend forecast

We had only compared the forecaster with "copy the last map". A later check scored the trend forecast in exactly the same way: same whole maps, same test parts, same score. We checked the check itself: scored separately on the raw files, copying's error matched the notebook's table to three decimals in all four tests, and the trend scores came out the same when the check was run inside the notebook. The trend forecast scored 0.91 to 0.99; the first version only 0.15 to 0.35, and −1.45 for late Australia. That also showed that our explanation for late Australia ("it never saw draining water") was wrong, because the training data did contain falling water. **Fix:** version 2, which starts from the trend forecast. It now scores the same as the trend forecast. We do not yet know whether starting from the trend, or counting the correction in centimetres, did the fixing.

12. Honest weaknesses

1. **At 30 minutes ahead a simple trend forecast does nearly all the work.** Version 2 only matches it. We have not shown that the CNN-LSTM adds anything at this horizon.
2. **The first version was weak, and we do not fully know why.** It was probably under-trained (the 30 m model kept its second-pass weights), and the very small numbers it had to predict may have made learning slow, but these are guesses. We changed two things at once when building version 2, so we cannot separate the causes.
3. **The capsule network is doing an easy job the hard way.** The rating in 30 minutes is nearly the rating now, so a simple rule on the forecast map gets 99%. It has not been compared with a plain output layer, and it is still built on the first version of the forecaster.
4. **Only 30 minutes ahead.** Useful warnings need several hours. The model's memory (the LSTM) has not been tested where it should matter more.
5. **Simulated data.** The floods are realistic simulations, not real measurements. Results on real floods could be different.
6. **Single runs and two test pairs.** Every result comes from one training run, and each map scale has only one training flood and one never-seen flood.
7. **The "typical error" looks smaller than it is** because it is averaged over the whole map, dry squares included.
8. **The water rule uses past rain as a guess for the next 30 minutes**, and the data follows the water balance only to within about 5%.

13. Next steps

1. **Forecast further ahead.** Feed each 30-minute forecast back in as input to reach several hours ahead, step by step. This is where the trend forecast should get worse and the model's memory has a real chance to matter, so compare against a trend forecast carried forward in the same way. We first need to decide what rain to assume for the future steps.
2. **Test what the model uses.** Remove the rain, or freeze the depth history, at test time. Replace the ConvLSTM with a plain convolution to see whether the memory itself helps.
3. **Give the capsule network a real job.** Rate the flooding 3 to 6 hours ahead, where "now" is no longer enough, and compare it with a plain output layer. Rebuild it, and the Grad-CAM pictures, on version 2.
4. **Repeat the runs** with different random starts, so we can see how much the numbers move by chance.

14. Questions the professor may ask, with simple answers

Why use a CNN?

Flood maps are pictures. A CNN is the standard tool for pictures: it learns to spot local patterns (water edges, slopes, rain patches) and recognises them anywhere on the map.

Why add an LSTM?

A single picture doesn't tell you whether water is rising or falling, and a flood is a sequence of maps. So the architecture we set out to build is a CNN with an LSTM, and we use a ConvLSTM so its memory keeps the shape of the map. We then tested it fairly. For a 30-minute forecast, continuing the trend already gets 91% to 99% of the possible improvement over copying, and the LSTM version adds only about 0.001 on top. Memory should matter more for longer forecasts, so that is what we test next.

Why not just use a bigger model?

We have not tested bigger models. We only have four floods, so we kept the model small (about 63,000 numbers); it trains in minutes. At 30 minutes ahead there is also little left to gain, at most about 1% to 9% of copying's error.

How do you know the model is actually good?

We compare it with two simple forecasts: "copy the last map" and "continue the trend". The first version beat copying but not the trend. Version 2 matches the trend. So it is as good as the strongest simple forecast we tried, and not yet better.

Why is the trend forecast so good?

Floods change smoothly: water that has been rising tends to keep rising for a while, and water that has been draining keeps draining. Over 30 minutes the next change is close to the last change. This is why we always compare with it.

Why predict only 30 minutes ahead?

It is the time between two maps in the data, and the simplest honest starting point. The results also show a limit: at 30 minutes the trend forecast already does almost everything, so longer forecasts (several hours) are where a learning model may have more to add. They are the next step.

Where is the NeuroSymbolic AI?

In two places. (1) A physics rule, conservation of water, corrects the forecaster's total amount of water. It helped the first version and makes almost no difference to version 2. (2) Three written if-then rules check the capsule network's rating and give a written reason for each decision. They raised its accuracy in every flood, partly because the rules use the flooded share of the map now and in the forecast, which is almost always a good guide to the rating.

Why a capsule network?

It keeps not just "how sure" but also "how it looks" (the direction of its output arrows), and its voting system lets parts of the tile agree on an answer. It works, but honestly, for a 30-minute rating a simple rule on the forecast map is better, and we have not compared it with a plain output layer. The plan is to give it a harder job.

What does Grad-CAM show?

Which parts of the map the model relied on. It lights up rivers, flood edges and rain patches, which are sensible places. It is an illustration; we have not tested how stable these pictures are.

Why does Australia score badly?

It did for the first version. That version added a little water over most of the map, which matters when the water hardly moves: at the end of that flood, copying is wrong by only about 1 mm. Version 2, which starts from the trend forecast, scores +0.989 there.

Is the model cheating by seeing future rain?

No. Each rain file covers the next 30 minutes, so we only give the model rain from earlier files, which has already fallen.

Why two models instead of one?

A 30 m square and a 480 m square behave very differently: water moves across them at very different speeds. One model for both would be confused.

15. Word list

Word	Plain meaning
Cell / square	One square of the map grid (30 m or 480 m wide)
Neural network	A program that learns a task from many examples instead of being told the steps
CNN	Convolutional neural network: a network for pictures that slides small pattern-detectors across the image
LSTM / ConvLSTM	A network with memory that reads things in order; the ConvLSTM's memory is a map
Training	Showing the network examples and nudging its numbers to make its answers less wrong
Validation	A held-back part of the data used during training to notice memorising
Test	Data never used in training, used only to measure the final result
Tile	A small square piece of the map (64 × 64 cells)
Copy the last map	The first yardstick forecast: "the water stays where it is"
Trend forecast	The second yardstick forecast: "the water keeps changing the way it just changed". It has one number and no network
Version 2	The CNN-LSTM that starts from the trend forecast and learns only a correction
Skill	How much of copying's error a forecast removes. Above 0 = better than copying, 1 = perfect
Capsule network	A network whose outputs are small arrows: length = how sure, direction = how the answer looks
Routing by agreement	The capsule network's voting: parts that agree get more say
Grad-CAM	A heat map showing which parts of the input most influenced the answer
NeuroSymbolic AI	Combining a learning network with written rules that people can read
Water balance	Water can't appear from nowhere: the total only grows by the rain that falls
Simulation	A computer re-creation of the flood using the laws of water flow

The numbers in this guide come from the outputs of the notebook *Flood_model_updated* (with the added comparison cells) and from the comparison with the trend forecast, scored on the same whole-map, 30-minute test.